MCP - Model Context Protocol

Table of Contents

Setup....2
Overview....3
  Versions....3
  Tool Using Models....4
MCP Tools and Resources....5
MCP Server Scripts for Tools....6
  System Health MCP....8
  Network Ping MCP....9
Calling Tools....11
MCP Resources....14
MCP Tool and Resource Calling....17
MCP Server with Bottle Web App Framework....18
MCP Server for Claude Desktop....19

This class teaches the basic concepts of building MCP (Model Context Protocol) Servers to provide AI systems tools. MCP is like a REST API for AI and anything you can write as a Python function can be accessed by your AI system.

Setup

Install Ollama

Ollama must be running on the local machine.for the projects to work.

```
Ollama.com
```

Pull a model with Tool Calling capabilities. IBM Granite4.1:3b should work on most laptop computers, but may have quirky results. gemma4:31b provides good results but requires a system with 32 GB of RAM.

You can also use MCP with API calls to AI services such as OpenAI's API.

```
ollama pull granite4.1:3b
```

```
python3 -m pip install ollama
```

Install MCP Module

```
python3 -m pip install mcp
```

Note:

Some Models with Tool calling capabilities may not work with the demonstrations shown in this class. Granite4.2:3b uses chain-of-thought which may confuse the model on how to implement tools.

```
DEMO SCRIPT USING granite4.2:3b vs 4.1
```

Overview

MCP or Model Context Protocol is like an API for AI systems. This is a client/server architecture where an AI systems asks the MCP server for a list of it's tools, and then picks one of those tools to use.

MCP Servers can also offer read only resources that can be used by AI Agents such as instructions, or documentation for answering queries.

MCP Servers can be connected to with Python scripts, or with applications such as Anthropic Claude Desktop.

MCP Servers can be full fledged network connected servers using a web app framework such as FastAPI, or can be a local Python script for your AI system to directly use without using TCP/IP.

Versions

MCP is currently on Version 2.0.

MCP was created by Anthropic and released November 2024.

Version 2 offers a number of improvements, and should be used instead of version 1.

Version 1 used FastMCP function and version 2 uses MCPServer.

MCP 1.0

```
from mcp.server.fastmcp import FastMCP
```

MCP 2.0

```
from mcp.server.mcpserver import MCPServer
```

MCP uses JSON-RPC 2.0 as its message format — requests have a method, optional params, and an id; responses have matching ids with either a result or error.

In MCP over HTTP, the client and server go through these steps, which match the server you just have:

1. **Initialize (client → server):** The client sends an `initialize` request with its protocol version and capabilities.
2. **Initialize response (server → client):** The server replies with its protocol version, its capabilities (here, `tools`), and its name and version.
3. **Initialized notification (client → server):** The client sends `notifications/initialized`. It has no `id`, so the server doesn't reply. This is why the code returns 202 for messages without an `id`.
4. **Discovery (client → server):** The client sends `tools/list`, and the server returns each tool's name, description, and input schema.
5. **Invocation (client → server):** When the model wants a tool, the client sends `tools/call` with the tool name and arguments.
6. **Result (server → client):** The server runs the tool and returns the output as `content`, such as `[{"type": "text", "text": "14:32:05"}]`.

Tool Using Models

You will have to use a model that explicitly states that it can use tools. Models can also have varying degrees of compatibility with MCP.

MCP Tools and Resources

An MCP Server is able to provide both Tools and Resources to AI Applications.

Tools are function calls that the AI System can make. After you make a request of your AI system it will query the MCP Server for a list of tools. You AI will then call the tool it finds approriate which will trigger the function on the MCP Server. The AI will then take the response from the servers function call and use it’s model to provide an answer to the user.

Resources are Read Only Data that the MCP Server can provide and will be called by the Application and not the AI LLM.

MCP Tool Example

```
@mcp.tool()
def get_time() -> str:
    """Return the current time (HH:MM:SS)."""
    return datetime.now().strftime("%H:%M:%S")
```

MCP Resource Example

```
@mcp.resource("hours://business")
def get_business_hours() -> str:
    """Provides the current hours of operation for the business."""
    return (
        "Store Hours of Operation:\n"
        "Monday - Friday: 9:00 AM - 6:00 PM\n"
        "Saturday: 10:00 AM - 4:00 PM\n"
        "Sunday: Closed"
    )
```

MCP Server Scripts for Tools

```
DEMO: mcp-basic-client.py
Run client script to that it can access tools for time and day of week.
```

To create tools for your MCP Server you import the MCPServer function from the mcp module. You then create functions with an mcp function decorator.

If you do not state a name for the tool then the name of the function will automatically be used. You can also add a description within a string inside the main function.

To explicitly set a name and description you can do so within the arguments for the tool decorator. The name should use snake case naming with additional punctuation such as this_is_a_really_cool_function.

```
messages = [
    {"role": "user", "content": "how long till midnight?"}
]
```

```
It is currently 13:37:13 (1:37 PM). Midnight would be at 24:00 or 0:00. So, there are approximately 12 hou
rs and 22 minutes left until midnight.
eli@lab MCP – Model Context Protocol %
```

```
python3 -m pip install mcp
```

mcp-server-basic.py

```
from datetime import datetime
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("time-server")

@mcp.tool()
def get_time() -> str:
    """Return the current time (HH:MM:SS)."""
    return datetime.now().strftime("%H:%M:%S")

@mcp.tool()
def get_day_month_year() -> str:
    """Return the current day of the week."""
    return datetime.now().strftime("%A, %B, %Y")

if __name__ == "__main__":
    mcp.run()
```

mcp-server-basic-full.py

```
from datetime import datetime
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("time-server")

@mcp.tool(
    name="Get_Current_Time",
    description="Return the current time in HH:MM:SS (24-hour) format, based on the
server's local clock.",
)
def get_time() -> str:
    return datetime.now().strftime("%H:%M:%S")

@mcp.tool(
    name="Get_Day_Month_And_Year",
    description="Return the current day of the week (e.g. 'Monday'), based on the
server's local clock.",
)
def get_day_month_year() -> str:
    return datetime.now().strftime("%A, %B, %Y")

if __name__ == "__main__":
    mcp.run()
```

System Health MCP

This MCP Server allows you query your AI about your system status. We have 3 tools that use OS level applications to pull data about your system. We use TOP to access data about CPU and RAM usage, PING to see the status of your internet connection and DF to see how much of your storage is used.

This eaxmple shows you how you can give access to your Operating System to your AI Application.

mcp-server-system-health.py

```
import subprocess

from mcp.server.mcpserver import MCPServer

mcp = MCPServer("System-Monitor")

@mcp.tool()
def get_system_health() -> str:
    """Return the output of the Top command to show CPU and Memory usage of
system"""
    result = subprocess.run(['top', '-l', '1', '-n', '0'], capture_output=True,
text=True)
    return result.stdout

@mcp.tool()
def get_network() -> str:
    """Return the Ping results from google.com"""
    result = subprocess.run(['ping', '-c', '1', 'google.com'], capture_output=True,
text=True)
    return result.stdout

@mcp.tool()
def get_storage() -> str:
    """Return the Utilization of the Local hard Drive"""
    result = subprocess.run(['df', '-h', '/'], capture_output=True, text=True)
    return result.stdout

if __name__ == "__main__":
    mcp.run()
```

Network Ping MCP

This example provides multiple tools that can be used by your client script. You can ask you script to “ping cnn.com” and it willuse multiple tools to solve the problem. It will pull out *cnn.com as the argument* for resolve_ip() to get it’s IP address, and then will use ping_ip() to ping the address. If you provide an IP Address then extract_ip() will try to get the IP to provide to the ping_ip() function.

mcp-server-ping.py

```
import ipaddress
import platform
import re
import socket
import subprocess
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("network-tools")

IPV4_PATTERN = re.compile(r"\b(?:\d{1,3}\.){3}\d{1,3}\b")

@mcp.tool()
def extract_ip(text: str) -> str:
    "Find and return the first valid IPv4 address in a block of text."
    for candidate in IPV4_PATTERN.findall(text):
        try:
            ipaddress.IPv4Address(candidate)  # rejects things like 999.1.1.1
            return candidate
        except ValueError:
            continue
    return "Error: no valid IPv4 address found in the text."

@mcp.tool()
def resolve_ip(hostname: str) -> str:
    "Resolve a hostname or domain (e.g. google.com) to its IPv4 address."
    try:
        return socket.gethostbyname(hostname)
    except socket.gaierror as e:
        return f"Error: could not resolve {hostname}: {e}"

@mcp.tool()
def ping_ip(ip: str, count: int = 4) -> str:
    "Ping an IP address and return the ping output."
    try:
        ipaddress.ip_address(ip)  # validate so arbitrary input never reaches the
shell
    except ValueError:
        return f"Error: {ip!r} is not a valid IP address."

    flag = "-n" if platform.system() == "Windows" else "-c"
    count = max(1, min(count, 10))
    try:
        proc = subprocess.run(
            ["ping", flag, str(count), ip],
            capture_output=True, text=True, timeout=30,
        )
        return proc.stdout or proc.stderr
    except subprocess.TimeoutExpired:
        return "Error: ping timed out."

if __name__ == "__main__":
    mcp.run()
```

Calling Tools

When you use your AI script to call tools it has the ability to call a number of tools at a time. So if you use the mcp-ping-server.py MCP Server you can ask it to ping cnn.com and it will use a tool to get the IP Address of cnn.com and then another to ping it.

You can even ask if cnn.com has better latency than fox.com and it will get the IP Addresses for both sites, pig both sites and then with the response will give you an answer.

In the for loop we set a maximum tool call of 10 in case it gets into an inescapable loop, and then the responses are appended to the message list so that the AI has access to all previous responses.

Note: Depending on what you're asking models may work well, or fail in odd ways. Granite4.1:3b can tell you the time, but if you ask how long till midnight the answer will be wildly off. Gemma4:31b works well for more complicated queries.

mcp-client.py

```
import asyncio
import ollama
import subprocess
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

# must support tool calling - gemma4:31b - granite4.1:3b
OLLAMA_MODEL = "gemma4:31b"
MCP_SERVER = "mcp-ping-server.py"
# mcp-server-basic-full.py
# mcp-server-system-health.py
# mcp-server-ping.py

server_params = StdioServerParameters(
    command="python3",
    args=[MCP_SERVER],  # path to your MCP server script
)

def mcp_tools_to_ollama_format(mcp_tools):
    tools = []
    for t in mcp_tools:
        tools.append({
            "type": "function",
            "function": {
                "name": t.name,
                "description": t.description or "",
                "parameters": t.input_schema or {"type": "object", "properties":
{}},
            },
        })
    return tools
```

```
async def query_ai(query):
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            # Discover available tools from the MCP server
            tools_result = await session.list_tools()
            ollama_tools = mcp_tools_to_ollama_format(tools_result.tools)

            ### ADD RESOURCE CODE HERE

            messages = [
                {
                    "role": "system",
                    "content":  "You are a helpful assistant with access to local
system tools and automation. "
                                "Use the provided tools to answer the user's
request whenever necessary. "
                                "Be brief"
                },
                {"role": "user", "content": query}
            ]
            for _ in range(10):

                response = ollama.chat(
                    model=OLLAMA_MODEL,
                    messages=messages,
                    tools=ollama_tools,
                )

                msg = response["message"]
                messages.append(msg)

                # If the model wants to call tools, execute them via MCP
                if msg.get("tool_calls"):
                    for call in msg["tool_calls"]:
                        name = call["function"]["name"]
                        args = call["function"]["arguments"]

                        result = await session.call_tool(name, arguments=args)
                        tool_output = "".join(
                            c.text for c in result.content if hasattr(c, "text")
                        )

                        messages.append({
                            "role": "tool",
                            "content": tool_output,
                            "name": name,
                        })

                    # Send tool results back to the model for a final answer
                    final = ollama.chat(model=OLLAMA_MODEL, messages=messages)
                    result = final["message"]["content"]
                    print('--- TOOLS')
```

```
                    print(ollama_tools)
                    print('--- TOOL CALL')
                    print(response)
                    print('--- TOOL RESPOSE')
                    print(tool_output)
                    print('--- LLM RESPONSE')
                    print(final)

                else:
                    result = msg["content"]
    return result

while True:
    query = input('Query About System: ')
    subprocess.run(['clear']) # Clear screen
    response = asyncio.run(query_ai(query))
    print(query)
    print(response)
```

```
--- TOOLS
[{'type': 'function', 'function': {'name': 'Get_Current_Time', 'description': "Return the current time in HH:MM:SS (24-hour) format
, based on the server's local clock.", 'parameters': {'properties': {}, 'type': 'object', 'title': 'get_timeArguments'}}}, {'type':
 'function', 'function': {'name': 'Get_The_Day_Month_And_Year', 'description': "Return the current day of the week (e.g. 'Monday'),
 based on the server's local clock.", 'parameters': {'properties': {}, 'type': 'object', 'title': 'get_day_month_yearArguments'}}}]
--- TOOL CALL
model='gemma4:31b' created_at='2026-10-09T18:10:59.694732Z' done=True done_reason='stop' total_duration=3975408250 load_duration=22
1817583 prompt_eval_count=124 prompt_eval_duration=126717000 eval_count=94 eval_duration=3589616000 message=Message(role='assistant
', content='', thinking='The user is asking for the time remaining until midnight. To answer this, I need to know the current time.
 I should look for a tool that provides the current time. The `Get_Current_Time` function seems appropriate for this purpose. After
 getting the current time, I can calculate the difference between it and midnight (00:00:00 of the next day).', images=None, tool_n
ame=None, tool_calls=[ToolCall(function=Function(name='Get_Current_Time', arguments={}))]) logprobs=None
--- TOOL RESPOSE
14:10:59
--- LLM RESPONSE
model='gemma4:31b' created_at='2026-10-09T18:11:15.295097Z' done=True done_reason='stop' total_duration=15574202584 load_duration=1
94370209 prompt_eval_count=137 prompt_eval_duration=461808000 eval_count=380 eval_duration=14899639000 message=Message(role='assist
ant', content='It is currently 9 hours, 49 minutes, and 1 second until midnight.', thinking='The current time is 14:10:59. I need t
o calculate how long until midnight (24:00:00).\n\nCalculation:\nMidnight = 24 hours, 0 minutes, 0 seconds.\nCurrent Time = 14 hour
s, 10 minutes, 59 seconds.\n\nDifference:\nHours: 23 - 14 = 9\nMinutes: 59 - 10 = 49\nSeconds: 60 - 59 = 1\n\nWait, let me do that
more carefully:\nTotal seconds in a day: $24 \\times 3600 = 86400$\nCurrent time in seconds since start of day: $(14 \\times 3600)
+ (10 \\times 60) + 59 = 50400 + 600 + 59 = 51059$\nSeconds remaining until midnight: $86400 - 51059 = 35341$\n\nConvert seconds ba
ck to H:M:S:\nHours: $\\lfloor 35341 / 3600 \\rfloor = 9$ hours.\nRemaining seconds: $35341 \\pmod{3600} = 2941$\nMinutes: $\\lfloo
r 2941 / 60 \\rfloor = 49$ minutes.\nSeconds: $2941 \\pmod{60} = 1$ second.\n\nSo, it is 9 hours, 49 minutes, and 1 second until mi
dnight.', images=None, tool_name=None, tool_calls=None) logprobs=None
--- RESPONSE
It is currently 9 hours, 49 minutes, and 1 second until midnight.
eli@lab MCP - Model Context Protocol %
```

MCP Resources

MCP Resources are Read Only Data that you can use to add additional context to your queries. Resources are called directly by the script and identified by their URI (hours://business).

For URI (Universal Resource Identifier) if you offer a string you can siply use word1://word2.

mcp-server-resource-hours.py

```
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("Business Hours Server")

@mcp.resource("hours://business")
def get_business_hours() -> str:
    """Provides the current hours of operation for the business."""
    return (
        "Store Hours of Operation:\n"
        "Monday - Friday: 9:00 AM - 6:00 PM\n"
        "Saturday: 10:00 AM - 4:00 PM\n"
        "Sunday: Closed"
    )

if __name__ == "__main__":
    mcp.run()
```

mcp-client-resource-hours.py

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    server_params = StdioServerParameters(
        command="python3",
        args=["mcp-server-resource-hours.py"] # Ensure server.py is in the same
folder
    )

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            print("--- Requesting Business Hours ---")

            resource_uri = "hours://business"
            result = await session.read_resource(resource_uri)

            content = result.contents[0].text
            print(f"Server responded with:\n\n{content}")

if __name__ == "__main__":
    asyncio.run(main())
```

```
--- Requesting Business Hours ---
Server responded with:

Store Hours of Operation:
Monday - Friday: 9:00 AM - 6:00 PM
Saturday: 10:00 AM - 4:00 PM
Sunday: Closed
eli@lab MCP - Model Context Protocol %
```

This script adds AI functionality to be able to ask questions and receive responses based on the resource. Be aware that different models are able to give better or worse answers.

mcp-resource-hours-ollama.py

```
import asyncio
import ollama
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    server_params = StdioServerParameters(
        command="python3",
        args=["mcp-server-resource-hours.py"]
    )

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            user_query = input("Ask a question about the business: ")

            resource_uri = "hours://business"
            resource_result = await session.read_resource(resource_uri)
            business_hours_context = resource_result.contents[0].text

            prompt = f"""
            You are a helpful business assistant. Use the following context to
answer the user question.
            If the answer isn't in the context, say you don't know.

            CONTEXT:
            {business_hours_context}

            USER QUESTION:
            {user_query}
            """
            response = ollama.chat(model='granite4.1:3b', messages=[
                {
                    'role': 'user',
                    'content': prompt,
                },
            ])

            print(response['message']['content'])

if __name__ == "__main__":
    asyncio.run(main())
```

```
 Ask a question about the business: are you open at 10pm on Saturday?
 No, you are not open at 10:00 PM on Saturday. The store hours for Saturday are 10:00 AM to 4:00 PM.
eli@lab MCP - Model Context Protocol %
```

MCP Tool and Resource Calling

You can add resources to your MCP Powered AI Script to provide context to the responses your scripts return. In this example we provide Qualtiy of Service values for RTC, Standard Internet and Bad Internet. The AI Script can the get the Latency from the Ping command and then compare the value against the resource values.

mcp-server-client-ping.py

```
            # ADD RESOURCE BLOCK AFTER OLLAMA_TOOLS AND BEFORE MESSAGES =
            # TO MCP-CLIENT.PY SCRIPT
            … … …
            ollama_tools = mcp_tools_to_ollama_format(tools_result.tools)

            resource_uri = "latency://quality"
            resource_result = await session.read_resource(resource_uri)
            context = resource_result.contents[0].text
            query = f'{query}\n CONTEXT \n {context}'

            messages = […… …. ….
```

mcp-server-ping.py

```
      # ADD TO END OF MCP-PING-SERVER.PY SCRIPT BEFORE RUN()

      @mcp.resource("latency://quality")
      def get_latency() -> str:
      """Provides quality of service for latency."""
      return (
            "> 30 ms is good for RTC\n"
            "30 to 90 ms is good for Standard Internet\n"
            "Over 100 ms is a bad connection\n"
      )
```

```
Yes, tacbell.com is good for VoIP. The average latency is 27.81 ms, which is under the
30 ms threshold for RTC.
Query About System:
```

MCP Server with Bottle Web App Framework

You can build an MCP Server that will provide Tools and Resources to internet/network clients using a Web App Framework such as Bottle. (We use Bottle generally for projects. You can use FastAPI, Django or any other web app framework)

For a client script to connect to it you will have to use the streamable_http_function. 3 lines have to be edited form the standard client script we have been using.

NOTE: You will have to start the server script in it's own terminal window, and the client script will have to run in a different one.

mcp-client-bottle.py

```
# from mcp.client.stdio import stdio_client
from mcp.client.streamable_http import streamable_http_client
…
MCP_SERVER = "http://localhost:8080/mcp"
# MCP_SERVER = "mcp-ping-server.py"
…
async def query_ai(query):
    async with streamable_http_client(MCP_SERVER) as (read, write):
    # async with stdio_client(server_params) as (read, write):
```

In the server script you will define you functions that will be avilable as MCP tools. You will then create a TOOLS{} dictionary that will provide information about the Tools to the client. In the for the handeer: key you will point it to the local function. You can also add lambda functions here.

mcp-server-bottle.py

```
from datetime import datetime, timezone
import json
from bottle import Bottle, request, response

app = Bottle()

SERVER_INFO = {"name": "bottle-time-date-mcp", "version": "1.0.0"}
PROTOCOL_VERSION = "2024-11-05"

# ---------------------------------------------------------------------------
# Tool implementations
# ---------------------------------------------------------------------------

def get_time(args):
    fmt = args.get("format", "%H:%M:%S")
    use_utc = args.get("utc", False)
    now = datetime.now(timezone.utc) if use_utc else datetime.now()
    return now.strftime(fmt)
```

```
def get_date(args):
    fmt = args.get("format", "%Y-%m-%d")
    use_utc = args.get("utc", False)
    now = datetime.now(timezone.utc) if use_utc else datetime.now()
    return now.strftime(fmt)

TOOLS = {
    "get_time": {
        "description": "Get the current time. Optionally pass a strftime "
                       "'format' string and/or 'utc': true.",
        "inputSchema": {
            "type": "object",
            "properties": {
                "format": {"type": "string", "description": "strftime format
string"},
                "utc": {"type": "boolean", "description": "Use UTC instead of local
time"},
            },
        },
        "handler": get_time,
    },
    "get_date": {
        "description": "Get the current date. Optionally pass a strftime "
                       "'format' string and/or 'utc': true.",
        "inputSchema": {
            "type": "object",
            "properties": {
                "format": {"type": "string", "description": "strftime format
string"},
                "utc": {"type": "boolean", "description": "Use UTC instead of local
time"},
            },
        },
        "handler": get_date,
    },
}

# ---------------------------------------------------------------------------
# JSON-RPC / MCP plumbing
# ---------------------------------------------------------------------------

def rpc_result(id_, result):
    return {"jsonrpc": "2.0", "id": id_, "result": result}

def rpc_error(id_, code, message):
    return {"jsonrpc": "2.0", "id": id_, "error": {"code": code, "message":
message}}
```

```
def handle_initialize(id_, params):
    return rpc_result(id_, {
        "protocolVersion": PROTOCOL_VERSION,
        "capabilities": {"tools": {}},
        "serverInfo": SERVER_INFO,
    })

def handle_tools_list(id_, params):
    tools = [
        {"name": name, "description": t["description"], "inputSchema":
t["inputSchema"]}
        for name, t in TOOLS.items()
    ]
    return rpc_result(id_, {"tools": tools})

def handle_tools_call(id_, params):
    name = params.get("name")
    args = params.get("arguments", {}) or {}
    tool = TOOLS.get(name)
    if tool is None:
        return rpc_error(id_, -32602, f"Unknown tool: {name}")
    try:
        text = tool["handler"](args)
    except Exception as exc:  # pragma: no cover
        return rpc_error(id_, -32000, f"Tool execution failed: {exc}")
    return rpc_result(id_, {"content": [{"type": "text", "text": text}]})

METHODS = {
    "initialize": handle_initialize,
    "tools/list": handle_tools_list,
    "tools/call": handle_tools_call,
}

@app.post("/mcp")
def mcp_endpoint():
    response.content_type = "application/json"
    try:
        payload = request.json
    except Exception:
        payload = None

    if not payload:
        return json.dumps(rpc_error(None, -32700, "Parse error"))

    id_ = payload.get("id")
    method = payload.get("method")
    params = payload.get("params", {}) or {}

    handler = METHODS.get(method)
    if handler is None:
        return json.dumps(rpc_error(id_, -32601, f"Method not found: {method}"))

    return json.dumps(handler(id_, params))
```

```
@app.get("/health")
def health():
    return {"status": "ok", "server": SERVER_INFO}

if __name__ == "__main__":
    app.run(host="localhost", port=8080, debug=True)
```

MCP Server for Claude Desktop

You can use MCP Server scripts for both Tools and Resources in Claude Desktop and other AI applications. They can be hosted remotely as a standard server service or local be accessed locally like a standard script.

You will have to edit the JSON configuration file to add a local MCP script to your Claude setup.

```
Open Claude Desktop, go to Claude → Settings → Developer → Edit Config. It opens
Finder at the file.
```

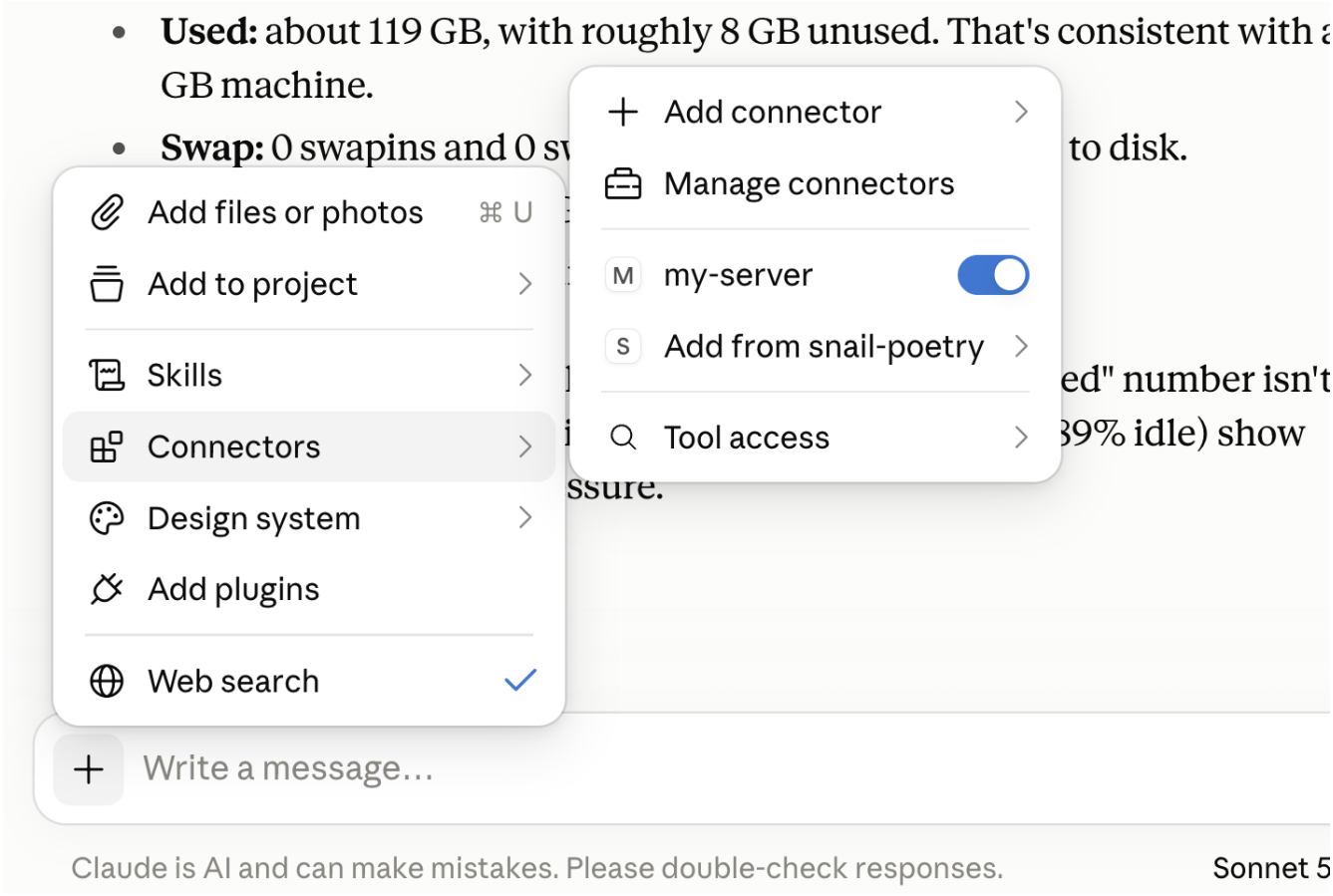
Direct path:

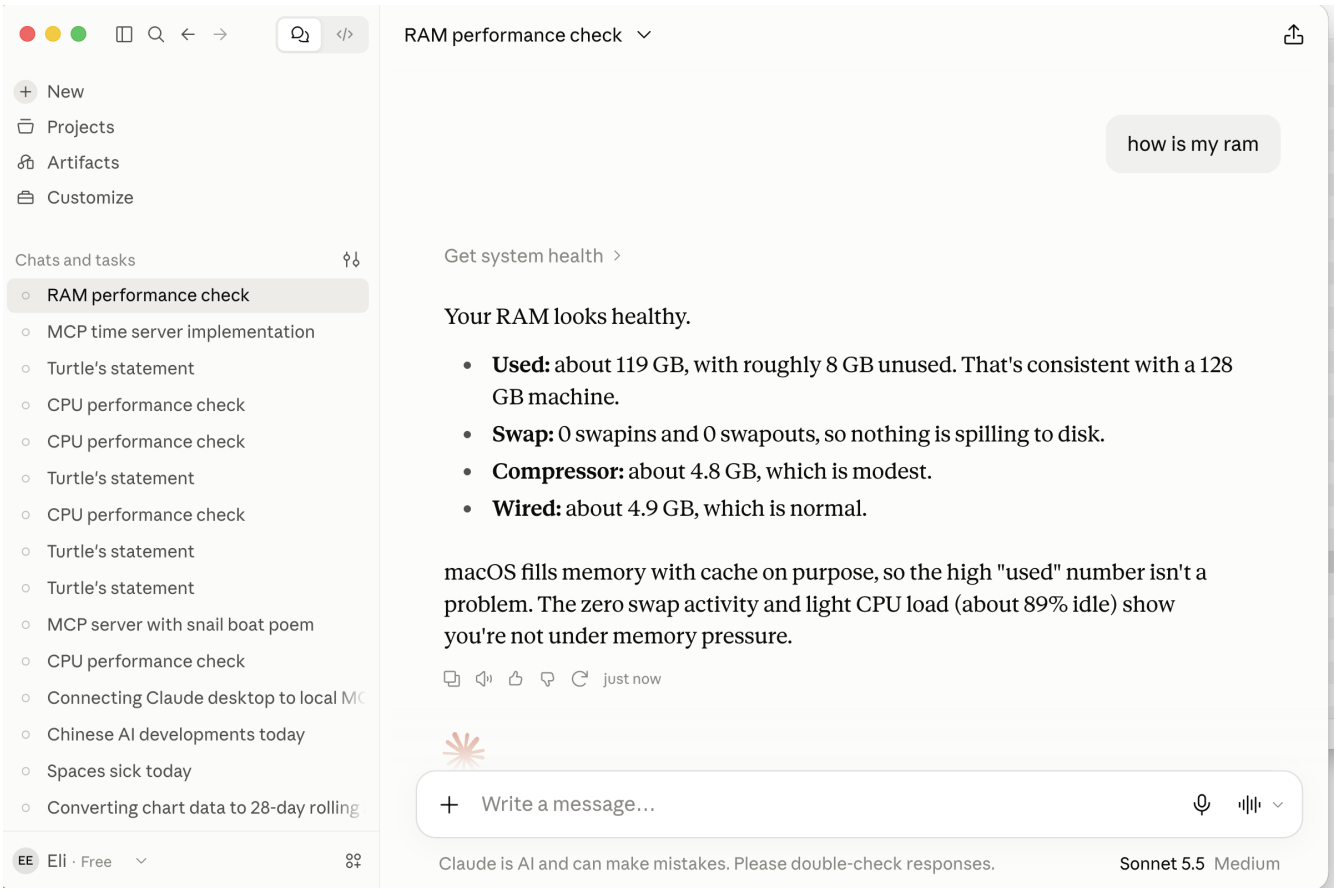
```
~/Library/Application Support/Claude/claude_desktop_config.json
```

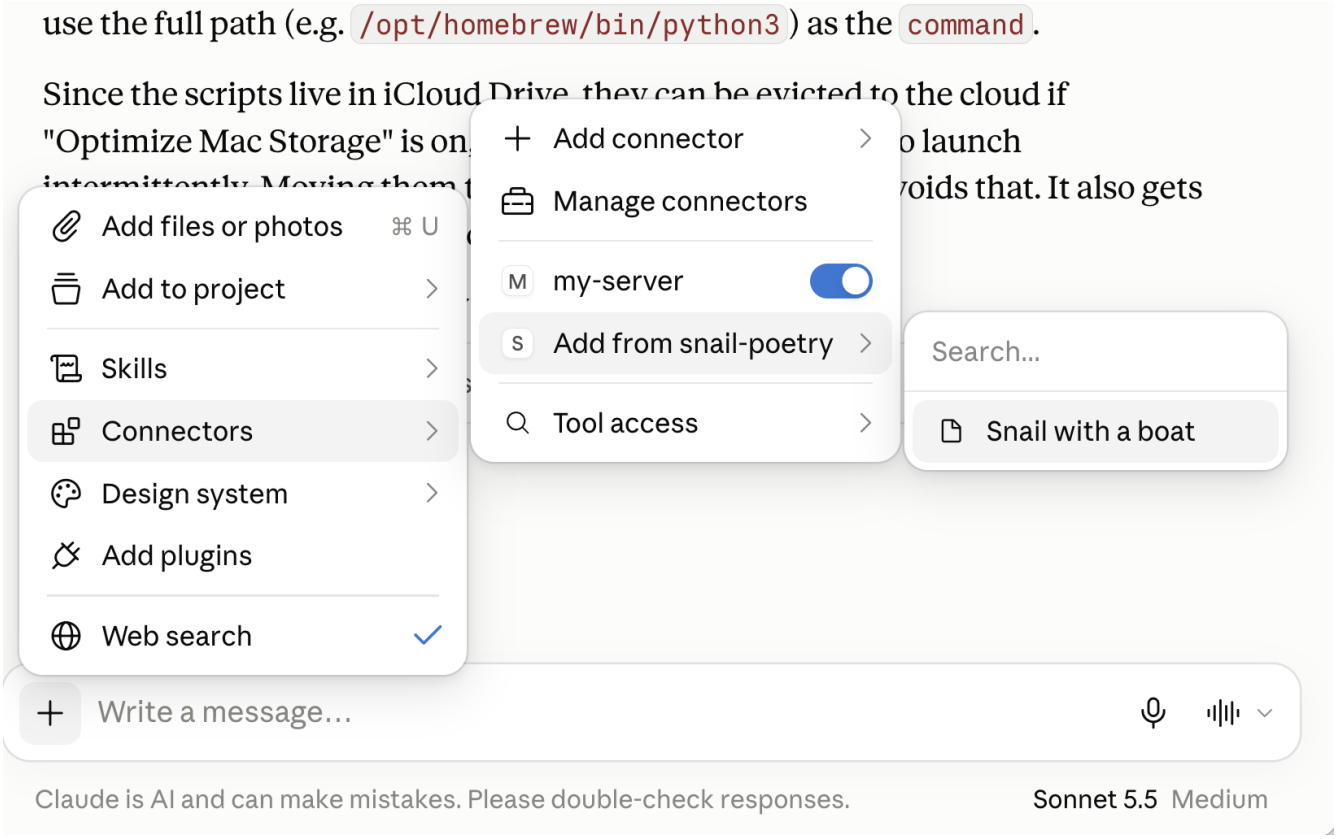
Add To Top of JSON file after first {

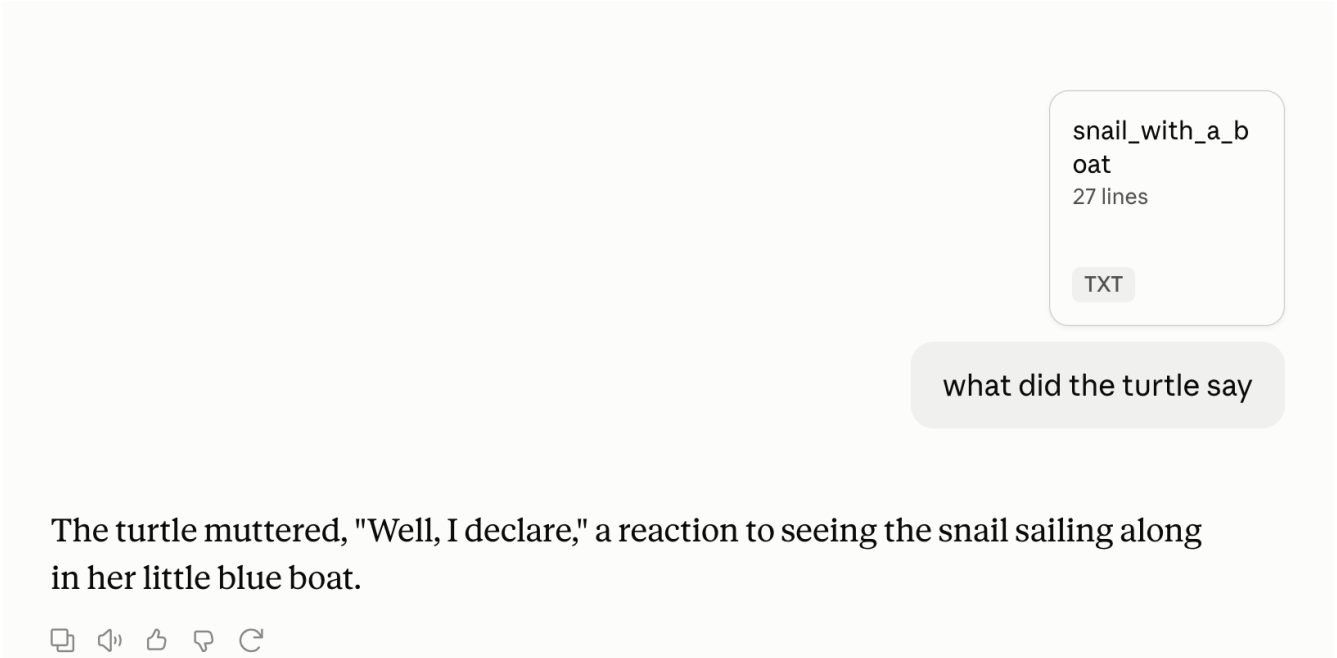
- my-server is name for MCP server
- command is for command to run Python
- args is for path to MCP script

```
  "mcpServers": {
    "my-server": {
      "command": "python3",
      "args": [
        "/Users/eli/Library/Mobile Documents/com~apple~CloudDocs/#Silicon
Dojo/#Projects/MCP/mcp-system-health-server.py"
      ]
    },
    "snail-poetry": {
      "command": "python3",
      "args": [
        "/Users/eli/Library/Mobile Documents/com~apple~CloudDocs/#Silicon
Dojo/#Projects/MCP/mcp-server-snail-test.py"
      ]
    }
  },
```


Used: about 119 GB, with roughly 8 GB unused. That's consistent with a
GB machine.
Swap: 0 swapins and 0 sw
to disk.
Add files or photos ⌘ U
Add to project
Skills
Connectors
Design system
Add plugins
Web search
Add connector
Manage connectors
M my-server
S Add from snail-poetry
Tool access
ed" number isn't
39% idle) show
ssure.
Write a message...
Claude is AI and can make mistakes. Please double-check responses.
Sonnet 5


New
Projects
Artifacts
Customize
Chats and tasks
RAM performance check
MCP time server implementation
Turtle's statement
CPU performance check
CPU performance check
Turtle's statement
CPU performance check
Turtle's statement
Turtle's statement
MCP server with snail boat poem
CPU performance check
Connecting Claude desktop to local MC
Chinese AI developments today
Spaces sick today
Converting chart data to 28-day rolling
EE Eli · Free
RAM performance check
how is my ram
Get system health
Your RAM looks healthy.
Used: about 119 GB, with roughly 8 GB unused. That's consistent with a 128 GB machine.
Swap: 0 swapins and 0 swapouts, so nothing is spilling to disk.
Compressor: about 4.8 GB, which is modest.
Wired: about 4.9 GB, which is normal.
macOS fills memory with cache on purpose, so the high "used" number isn't a problem. The zero swap activity and light CPU load (about 89% idle) show you're not under memory pressure.
just now
Write a message...
Claude is AI and can make mistakes. Please double-check responses.
Sonnet 5.5 Medium


use the full path (e.g. /opt/homebrew/bin/python3) as the command.
Add files or photos ⌘ U
Add to project
Skills
Connectors
Design system
Add plugins
Web search
Add connector
Manage connectors
M my-server
S Add from snail-poetry
Tool access
Search...
Snail with a boat
Write a message...
Claude is AI and can make mistakes. Please double-check responses.
Sonnet 5.5 Medium


snail_with_a_boat
27 lines
TXT
what did the turtle say
The turtle muttered, "Well, I declare," a reaction to seeing the snail sailing along in her little blue boat.

To Add a Remote MCP Server

